



RISC-V Hart to Trace Interface Specification

Authors: Iain Robertson

Version v0.91, 2026-08-07: Frozen

Table of Contents

Preamble	1
Copyright and license information	2
Contributors	3
1. Introduction	4
1.1. Terminology	4
1.2. Nomenclature	5
2. RISC-V Hart to Trace Encoder interface	6
2.1. Instruction Trace Interface requirements	6
2.1.1. Jump classification and target inference	7
2.1.2. Relationship between RISC-V core and the encoder	8
2.2. Instruction Trace Interface	8
2.2.1. Simplifications for single-retirement	13
2.2.2. Alternative multiple-retirement interface configurations	13
2.2.3. Optional sideband signals	14
2.2.4. Using trigger outputs from the Debug Module	14
Considerations for triggering from multi-memory access instructions	15
2.2.5. Example retirement sequences	15
2.3. Data Trace Interface requirements	16
2.4. Data Trace Interface	16
2.4.1. Multi-memory-access Instructions	19
3. Hart to Trace Encoder interface extensions	21
3.1. Trace State	21
3.1.1. External Debug Security (sdsec)	21
3.1.2. Unbounded Wait	21
3.1.3. Backwards compatibility considerations	22

Preamble



This document is in the [Frozen state](#)

*This document is in the Freeze state. Changes are unlikely at this point of the specification.
Visit riscv.org/spec-state for further details.*

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2026 by RISC-V International.

This document is a derivative of [Efficient Trace for RISC-V Specification](#), June 16, 2025, edited by Gajinder Panesar and Iain Robertson, released under the Creative Commons Attribution 4.0 International License ©2025.

Chapter 3 includes extensions to this.

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Iain Robertson <iain.robertson@siemens.com> - author
- Robert Chyla (MIPS)
- Ved Shanbhogue (Meta)

Chapter 1. Introduction

The RISC-V Hart to Trace Encoder Interface (RHTI) was originally developed (and ratified) as part of the [Efficient Trace for RISC-V \(E-Trace\) Specification](#). However, the interface itself is not E-Trace specific, and is suitable for use with any instruction and data trace encoding protocol. As such, this document makes it available as a standalone specification.

This specification also includes a normative change related to the handling of multi-memory access instructions - see [Section 2.4.1](#).

1.1. Terminology

The following terms have a specific meaning in this specification.

- **branch**: an instruction which conditionally changes the execution flow
- **CSR**: control/status register
- **decoder**: a piece of software that takes the trace packets emitted by the encoder and reconstructs the execution flow of the code executed by the RISC-V hart
- **discontinuity**: a change in the program counter that is anything other than an automatic increment to the next instruction located contiguously in memory
 - **inferable discontinuity**: a discontinuity in the program counter resulting from a branch or **direct jump**
 - **uninferable discontinuity**: a discontinuity in the program counter that is not inferable (see above)
 - **updiscon**: contraction of 'uninferable PC discontinuity'
- **E-Trace**: Abbreviation of *Efficient Trace for RISC-V*
- **ELF**: executable and linkable format
- **encoder**: a piece of hardware that takes in instruction execution information from a RISC-V hart and transforms it into trace packets
- **exception**: an unusual condition occurring at run time associated with an instruction in a RISC-V hart
- **hart**: a RISC-V hardware thread
- **final instruction**: the instruction retired immediately before tracing stops.
- **last instruction**: in a block of sequentially retired instructions, the instruction at the end of the sequence
- **interrupt**: an external asynchronous event that may cause a RISC-V hart to experience an unexpected transfer of control
- **ISA**: instruction set architecture
- **jump**: an instruction which unconditionally changes the execution flow
 - **direct jump**: an instruction which unconditionally changes the execution flow by changing the PC by a constant value
 - **indirect jump**: an instruction which unconditionally changes the execution flow by changing the PC to a computed value
 - **inferable jump**: a jump where the target address is supplied via a constant embedded within the jump opcode
 - **uninferable jump**: a jump which is not inferable (see above)

- **LSB**: least significant bit
- **MSB**: most significant bit
- **Multi-memory access (MMA) instruction**: An instruction such as those introduced by the Zcmp and Vector extensions that results in multiple loads or stores
- **packet**: the atomic unit of encoded trace information emitted by the encoder
- **PC**: program counter
- **program counter**: a register containing the address of the instruction being executed
- **retire**: the final stage of executing an instruction, when the machine state is updated and the **minstret** CSR is updated (sometimes referred to as 'commit' or 'graduate')
- **RHTI RISC-V Hart to Trace Encoder Interface** (this specification)
- **trace decoder**: see **decoder**
- **trace encoder**: see **encoder**
- **trap**: the transfer of control to a trap handler caused by either an exception or an interrupt

1.2. Nomenclature

In the following sections items in **bold** are signals or fields within a packet.

Items in ***bold italics*** are mnemonics for instructions defined in the RISC-V ISA

Items **lowLighted** are CSRs or other registers defined in the RISC-V ISA

Items in *italics* with names ending '*_p*' refer to parameters either built into the hardware or configurable hardware values.

Chapter 2. RISC-V Hart to Trace Encoder interface

2.1. Instruction Trace Interface requirements

This section describes in general terms the information which must be passed from the RISC-V hart to the trace encoder for the purposes of Instruction Trace, and distinguishes between what is mandatory, and what is optional.

The following information is mandatory:

- The number of instructions that are being retired;
- Whether there has been an exception or interrupt, and if so the cause (from the `scause/vscause/mcause` etc. CSR) and trap value (from the `stval/vstval/mtval` etc. CSR).

The register set to output should be the set that is updated as a result of the exception (i.e. the set associated with the privilege level immediately following the exception);

- The current privilege level of the RISC-V hart;
- The instruction type of retired instructions for:
 - Jumps with a target that cannot be inferred from the source code;
 - Taken and non-taken branches;
 - Return from exception or interrupt (**ret* instructions).
- The instruction address for:
 - Jumps with a target that *cannot* be inferred from the source code;
 - The instruction retired immediately after a jump with a target that *cannot* be inferred from the source code (also referred to as the target or destination of the jump);
 - Taken and non-taken branches;
 - The last instruction retired before an exception or interrupt;
 - The first instruction retired following an exception or interrupt;
 - The last instruction retired before a privilege change;
 - The first instruction retired following a privilege change.

The following information is optional:

- Context or Time information:
 - The context and/or hart ID and/or time;
 - The type of action to take when context or time data changes.
- The *instruction_type* of instructions for:
 - Calls with a target that *cannot* be inferred from the source code;
 - Calls with a target that *can* be inferred from the source code;
 - Other jumps without linkage with a target that *cannot* be inferred from the source code;
 - Other jumps without linkage with a target that *can* be inferred from the source code;
 - Returns with a target that *cannot* be inferred from the source code;
 - Returns with a target that *can* be inferred from the source code;

- Co-routine swap;
- Other jumps with linkage which don't fit any of the above classifications with a target that *cannot* be inferred from the source code;
- Other jumps with linkage which don't fit any of the above classifications with a target that *can* be inferred from the source code.
- If context or time is supported then the *instruction_address* for:
 - The last instruction retired before a context or a time change;
 - The first instruction retired following a context or time change.
- Whether jump targets are sequentially inferable or not.

The mandatory information is the bare-minimum required to implement a basic branch trace encoding algorithm. The optional information facilitates alternative or improved trace algorithms:

- Implicit return mode does not usually generate trace packets for the return addresses of functions. In order to do this the encoder needs to be aware of function calls and returns (see section 3.2.3 of the [Efficient Trace for RISC-V Specification](#), or section 9.2 of the [RISC-V N-Trace Specification](#));
- A simpler algorithm useful for basic code profiling would only report function calls and returns, again regardless of whether the target can be inferred or not;
- Branch prediction techniques can be used to further improve the encoder efficiency, particularly for loops. This requires the encoder to be aware of the address of all branches, whether they are taken or not.
- Uninferable jumps can be treated as inferable (which don't need to be reported in the trace output) if both the jump and the preceding instruction which loads the target into a register have been traced.

2.1.1. Jump classification and target inference

Jumps are classified as *inferable* or *uninferable*. An *inferable* jump has a target which can be deduced from the binary executable or representation thereof (e.g. ELF). For the purposes of this specification, the following strict definition applies:

If the target of a jump is supplied via a constant embedded within the jump opcode, it is classified as *inferable*. Jumps which are not *inferable* are by definition *uninferable*.

However, there are some jump targets which can still be deduced from the binary executable by considering pairs of instructions even though by the above definition they are classified as *uninferable*. Specifically, when the source register for the jump instruction is supplied via

- an *lui* or *c.lui* (a register which contains a constant), or
- an *auipc* (a register which contains a constant offset from the PC).

Such jump targets are classified as *sequentially inferable* if the pair of instructions are retired consecutively (i.e. the *auipc*, *lui* or *c.lui* immediately precedes the jump). Note: the restriction that the instructions are retired consecutively is necessary in order to minimize the additional signalling needed between the hart and the encoder, and should have a minimal impact on trace efficiency as it is anticipated that consecutive execution will be the norm. Support for sequentially inferable jumps is optional.

Jumps may optionally be further classified according to the recommended calling convention into *Calls*, *Returns*, *Co-routine swap*, and *other jumps with and without linkage*. Full details are provided in [Table 5](#).

2.1.2. Relationship between RISC-V core and the encoder

The encoder is intended to encode the instructions executed on a single hart.

It is however commonplace for a RISC-V core to contain multiple harts. This can be supported by the core in several different ways:

- Implement a separate instance of the interface per hart. Each instance can be connected to a separate encoder instance, allowing all harts to be traced concurrently. Alternatively, external muxing may be used in conjunction with a single encoder in order to trace one particular hart at a time.
- Implement a single interface for the core, with muxing inside the core to select which hart to connect to the interface.

(Whilst it is technically feasible to use a single encoder with multiple harts operating in a fine-grained multi-threaded configuration, the frequent context changes that would occur as a result of thread-switching would result in extremely poor encoding efficiency, and so this configuration is not recommended.)

2.2. Instruction Trace Interface

This section describes the interface between a RISC-V hart and the trace encoder that conveys the information described in [Section 2.1](#). Signals are assigned to one of the following groups:

- M: Mandatory. The interface must include an instance of this signal.
- O: Optional. The interface may include an instance of this signal.
- MR: Mandatory, may be replicated. For harts that can retire a maximum of N "special" instructions per clock cycle, the interface must include N instances of this signal.
- OR: Optional, may be replicated. For harts that can retire a maximum of N "special" instructions per clock cycle, the interface must include zero or N instances of this signal.
- BR: Block, may be replicated. Mandatory for harts that can retire multiple instructions in a block. Replication as per OR. If omitted, the interface must include SR group signals instead.
- SR: Single, may be replicated. Mandatory for harts that can only retire one instruction in a block. Replication as per OR (see [Section 2.2.2](#)). If omitted, the interface must include BR group signals instead.

"Special" instructions are those that require **itype** to be non-zero.

Table 1. Instruction interface signals

Signal	Group	Function
itype [<i>itype_width_p-1:0</i>]	MR	Termination type of the instruction block. Encoding given in Table 4 and Table 5 .
cause [<i>ecause_width_p-1:0</i>]	M	Exception or interrupt cause (scause / vscause / mcause). Ignored unless itype =1 or 2.
tval [<i>iaddress_width_p-1:0</i>]	M	The associated trap value, e.g. the faulting virtual address for address exceptions, as would be written to the stval / vstval / mtval CSR. Future optional extensions may define tval to provide ancillary information in cases where it currently supplies zero. Ignored unless itype =1.
priv [<i>privilege_width_p-1:0</i>]	M	Privilege level for all instructions retired on this cycle. Encoding given in Table 6 . Codes 4-7 optional.

Signal	Group	Function
iaddr [<i>iaddress_width_p</i> -1:0]	MR	The address of the 1st instruction retired in this block. Invalid if iretire =0 unless itype =1, in which case it indicates the address of the instruction which incurred the exception.
context [<i>context_width_p</i> -1:0]	O	Context for all instructions retired on this cycle.
time [<i>time_width_p</i> -1:0]	O	Time generated by the core.
ctype [<i>ctype_width_p</i> -1:0]	O	Reporting behavior for context . Encoding given in Table Table 7 . Codes 2-3 optional.
sijump	OR	If itype indicates that this block ends with an uninferable discontinuity, setting this signal to 1 indicates that it is sequentially inferable and may be treated as inferable by the encoder if the preceding <i>auipc</i> , <i>lui</i> or <i>c.lui</i> has been traced. Ignored for itype codes other than 6, 8, 10, 12 or 14.

[Table 1](#) and [Table 2](#) list the signals in the interface designed to efficiently support retirement of multiple instructions per cycle. The following discussion describes the multiple-retirement behavior. However, for harts that can only retire one instruction at a time, the signalling can be simplified, and this is discussed subsequently in [Section 2.2.1](#).

Table 2. Instruction interface signals - multiple retirement per block

Signal	Group	Function
iretire [<i>iretire_width_p</i> -1:0]	BR	Number of halfwords represented by instructions retired in this block.
ilastsize [<i>ilastsize_width_p</i> -1:0]	BR	The size of the last retired instruction is $2^{\text{ilastsize}}$ half-words. Ignored if iretire is 0.

Table 3. Instruction interface signals - single retirement per block

Signal	Group	Function
iretire [0:0]	SR	Number of instructions retired in this block (0 or 1).
ilastsize [<i>ilastsize_width_p</i> -1:0]	SR	The size of the last retired instruction in this block is $2^{\text{ilastsize}}$ half-words. Ignored if iretire is 0.

Table 4. Instruction Type (**itype**) encoding (*itype_width_p* = 3)

Value	Name	Instruction	Condition/Notes
0	Other		Last instruction in the block matches none of the other instructions defined in this table
1	Exception		An exception that traps occurred following the last instruction. Note: ECALL, EBREAK or C.EBREAK instructions raise exceptions but do not retire
2	Interrupt		An interrupt that traps occurred following the last retired instruction
3	Trap return	MRET SRET	Return from exception or interrupt handler
4	Non-taken branch	Conditional branch	Not-taken direct, conditional branch
5	Taken branch	Conditional branch	Taken direct, conditional branch

Value	Name	Instruction	Condition/Notes
6	Uninferable jump Return	JALR rd, rs C.JALR rs C.JR rs CM.POPRET CM.POPRETZ	Uninferable jump Defined by Zcmp extension
7	reserved		

Table 5. Instruction Type (**itype**) encoding (*itype_width_p* = 4)

Value	Name	Instruction	Condition/Notes
0	Other		Last instruction in the block matches none of the other instructions defined in this table
1	Exception		An exception that traps occurred following the last instruction. Note: ECALL, EBREAK or C.EBREAK instructions raise exceptions but do not retire
2	Interrupt		An interrupt that traps occurred following the last retired instruction
3	Trap return	MRET SRET	Return from exception or interrupt handler
4	Non-taken branch	Conditional branch	Not-taken direct, conditional branch
5	Taken branch	Conditional branch	Taken direct, conditional branch
6	reserved		
7	reserved		
8	Uninferable call	JALR rd, rs C.JALR rs	rd = link and rs != link rd = link and rs = link and rd = rs rs != x5
9	Inferable call	JAL rd C.JAL CM.JALT	rd = link Expands to: JAL x1, offset Defined by Zcmp extension
10	Uninferable jump (without linkage)	JALR rd, rs C.JR rs	rd = x0 and rs != link rs != link
11	Inferable jump	JAL rd C.J CM.JT	rd = x0 Expands to: JAL x0, offset Defined by Zcmp extension
12	Co-routine swap	JALR rd, rs C.JALR rs	rd = link and rs = link and rd != rs rs = x5
13	Return	JALR rd, rs C.JR rs CM.POPRET CM.POPRETZ	rd != link and rs = link rs = link Defined by Zcmp extension

Value	Name	Instruction	Condition/Notes
14	Other uninferable jump (with linkage)	JALR rd, rs	rd != link and rd != x0 and rs != link
15	Other inferable jump (without linkage)	JAL rd	rd != link and rd != x0



Branches (**itype**=4, 5) are always conditional, direct branches. In RISC-V ISA all jumps, calls, returns are always unconditional.



Symbol **link** means register **x1** or **x5** as specified in The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA document.



Jump instructions (**CM.JT** and **CM.JALT**) defined by ratified **Zcmp** extension are handled as direct (inferable) jumps as jump tables are assumed to be static and known to the trace decoder.

Table 6. Privilege level (**priv**) encoding

Value	Description
0	U
1	S/HS
2	reserved
3	M
4	D (debug mode)
5	VU
6	VS
7	reserved

The information presented in a block represents a contiguous sequence of instructions starting at **iaddr**, all of which retired in the same cycle. Note if **itype** is 1 or 2 (indicating an exception or an interrupt), the number of instructions retired may be zero. **cause** is only defined if **itype** is 1 or 2, and **tval** if **itype** is 1. If **iretire**=0 and **itype**=0, the values of all other signals are undefined.

iretire contains the number of (16-bit) half-words represented by instructions retired in this block, and **ilastsize** the size of the last instruction. Half-words rather than instruction count enables the encoder to easily compute the address of the last instruction in the block without having access to the size of every instruction in the block.

itype can be 3 or 4 bits wide. If *itype_width_p* is 3, a single code (6) is used to indicate all uninferable jumps. This is simpler to implement, but precludes use of the implicit return mode (see [Implicit Return](#)), which requires jump types to be fully classified. Note that when *itype_width_p* is 3, **itype** = 0 is used for inferable calls. However, inferable calls must still be the last instruction retired in a block, otherwise the block would not be comprised of contiguous instructions.

Whilst **iaddr** is typically a virtual address, it does not affect the encoder's behavior if it is a physical address.

For harts that can retire a maximum of N non-zero **itype** values per clock cycle, the signal groups MR, OR and either BR or SR must be replicated N times. Typically N is determined by the maximum number of branches that can be retired per clock cycle. Signal group 0 represents information about the oldest instruction block, and group N-1 represents the newest instruction block. The interface supports no more than one privilege change, context change, exception or interrupt per cycle and so signals in groups M and O are not replicated. Furthermore, **itype** can only take the value 1 or 2 in one of the signal groups, and this must be the newest valid group (i.e. **iretire** and **itype** must be zero for higher numbered groups). If fewer than N groups are required in a cycle, then lower numbered groups must be used first. For example, if there is one branch, use only group 0, if there are two branches, instructions up to the 1st branch must be reported in group 0 and instructions up to the 2nd branch must be reported in group 1 and so on.

sijump is optional and may be omitted if the hart does not implement the logic to detect sequentially inferable jumps. If the encoder offers an **sijump** input it must also provide a parameter to indicate whether the input is connected to a hart that implements this capability, or tied off. This is to ensure the trace decoder can be made aware of the hart's capability. Enabling sequentially inferable jump mode in the encoder and trace decoder when the hart does not support it will prevent correct reconstruction by the trace decoder.

The **context** and/or the **time** field can be used to convey any additional information to the trace decoder. For example:

- The address space and virtual machine IDs (**ASID** and **VMID** respectively). Where present it is

recommended these values be wired to bits [15:0] and [29:16];

- The software thread ID;
- The process ID from an operating system;
- It could be used to convey the values of CSRs to the trace decoder by setting **context** to the CSR number and value when a CSR is written;
- In cases where a single encoder is being shared amongst multiple harts (see [Section 2.1.2](#)), it could also be used to indicate the hart ID, in cases where the hart ID can be changed dynamically.
- Time from within the hart

[Table 7](#) specifies the actions for the various **ctype** values. A typical behavior would be for this signal to remain zero except on the 1st retirement after a context change or when a time value should be reported. Note also that **context** and **time** are automatically included in trace packets reporting the 1st instruction of a trap handler, or after a privilege change, and so this signal can remain zero in these cases as well (though it is not required). *ctype_width_p* may be 1 or 2. The reduced width option only provides support for reporting context changes imprecisely.

Table 7. Context type ctype values and corresponding actions

Type	Value	Actions
Unreported	0	No action (don't report context).
Report context imprecisely	1	An example would be a SW thread or operating system process change. Report the new context value at the earliest convenient opportunity. It is reported without any address information, and the assumption is that the precise point of context change can be deduced from the source code (e.g. a CSR write).
Report context precisely	2	Report the address of the 1st instruction retired in this block, and the new context. If there were unreported branches beforehand, these need to be reported first. Treated the same as a privilege change.
Report context as an asynchronous discontinuity	3	An example would be a change of hart. Need to report the last instruction retired on the previous context, as well as the 1st on the new context. Treated the same as an exception.

2.2.1. Simplifications for single-retirement

For harts that can only retire one instruction at a time, the interface can be simplified to the signals listed in [Table 1](#) and [Table 3](#). The simplifications can be summarized as follows:

- **iretire** simply indicates whether an instruction retired or not;

Note: **ilastsize** is still needed in order to determine the address of the next instruction, as this is the predicted return address for [Implicit Return mode](#).

The parameter *retires_p* which indicates to the encoder the maximum number of instructions that can be retired per cycle can be used by an encoder capable of supporting single or multiple retirement to select the appropriate interpretation of **iretire**.

2.2.2. Alternative multiple-retirement interface configurations

For a hart that can retire multiple instructions per cycle, but no more than one branch, the preferred solution is to use one instance of signals from groups BR, MR and OR. However, if the hart can retire N branches in a cycle, N instances of signals from groups MR, OR and either SR or BR must be used (each instance can be either a single instruction or a block).

If the hart can retire N instructions per cycle, but only one branch, it is allowed (though not recommended) to provide explicit details of every instruction retired by using N instances of signals from groups SR, MR and OR.

2.2.3. Optional sideband signals

Optional sideband signals may be included to provide additional functionality, as described in [Table 8](#) and [Table 9](#).

Note, any user defined information that needs to be output by the encoder will need to be applied via the **context** input.

Table 8. Optional sideband encoder input signals

Signal	Group	Function
<code>impdef[impdef_width_p-1:0]</code>	O	Implementation defined sideband signals. A typical use for these would be for filtering.
<code>trigger[2+:0]</code>	[1:0]: O [2+]: OR	A pulse on bit 0 will cause the encoder to start tracing, and continue until further notice, subject to other filtering criteria also being met. A pulse on bit 1 will cause the encoder to stop tracing until further notice. See Section 2.2.4).
<code>halted</code>	O	Hart is halted. Upon assertion, the encoder will output a packet containing the information the trace decoder requires in order to determine the final instruction retired before halting (e.g. the address of that instruction, or the instruction count since the last reported instruction), then indicate that tracing has stopped. Upon deassertion, the encoder will start tracing again, commencing with a synchronization packet. Note: If this signal is not provided, it is strongly recommended that Debug mode can be signalled via the priv signal. This will allow tracing in Debug mode to be controlled via the optional filtering capabilities. (This signal has been superseded by the Trace State extension.)
<code>reset</code>	O	Hart is in reset. Provided the encoder is in a different reset domain to the hart, this allows the encoder to indicate that tracing has ended on entry to reset, and restarted on exit. Behavior is as described above for halt.

Table 9. Optional sideband encoder output signals

Signal	Group	Function
<code>stall</code>	O	Stall request to hart. Some applications may require lossless trace, which can be achieved by using this signal to stall the hart if the trace encoder is unable to output a trace packet (for example due to back-pressure from the packet transport infrastructure).

2.2.4. Using trigger outputs from the Debug Module

The debug module of the RISC-V hart may have a trigger unit. This defines several trigger CSRs (for example `mcontrol6`, `icount`, etc.) that contain a 4-bit **action** field. Codes 2 - 5 of this field are for trace use. These action codes are hereby defined as shown in [table Table 10](#). If implemented, each action must generate a pulse on an output from the hart, on the same cycle as the instruction which caused the trigger is retired.

Table 10. Debug Module trigger support (action)

Value	Description
2	<i>Trace-on</i> . This should be connected to <code>trigger[0]</code> if the encoder provides it.

Value	Description
3	<i>Trace-off</i> . This should be connected to trigger[1] if the encoder provides it.
4	<i>Trace-notify</i> . This should be connected to trigger[1 + blocks:2] if the encoder provides it. This will cause the encoder to output a packet containing the address of the last instruction in the block if it is enabled. One bit per block.

Trace-on and Trace-off actions provide a means for the hart to control when tracing starts and stops. It is recommended that tracing starts from the oldest instruction retired in the cycle that Trace-on is asserted, and stops following the newest instruction retired in the cycle that Trace-off is asserted (subject to any optional filtering).

It follows from this that:

- if tracing is enabled and trace-off occurs on the cycle before trace-on, then tracing will continue unimpeded (i.e. it stays on);
- if tracing is disabled and trace-on and trace-off triggers occur simultaneously then only the instructions retired in that cycle will be traced.

Trace-notify provides means to ensure that a specified instruction is explicitly reported (subject to any optional filtering). This capability is sometimes known as a watchpoint.

Considerations for triggering from multi-memory access instructions

Debug module triggers may activate from one or more of the loads or stores initiated by an MMA instruction. Furthermore, MMA instructions may be interrupted or faulted before completion, and after returning from the trap may resume where they left off, or re-execute in their entirety. However, the **trigger** signals are only considered when an instruction retires.

- For an MMA instruction that executes to retirement without interruption, the hart must ensure all triggers initiated during the MMA instruction are output on the trigger signals when the instruction retires
- For an MMA instruction that traps part way through, and which will re-execute from the beginning on returning from the trap, any triggers occurring before the trap must be discarded (they will re-occur when the instruction re-executes). This applies to all Zcmp instructions.
- For an MMA instruction that traps part way through, and which is likely to resume from where it left off on returning from the trap, any triggers occurring before the trap must be output along with the trap, as there is no mechanism for the hart to store this trigger state.

There are also considerations for the trace encoder when starting trace via a *trace-on* trigger on an MMA instruction - see [Section 2.4.1](#).

2.2.5. Example retirement sequences

Table 11. Example 1: 9 Instructions retired over four cycles, 2 branches

Retired	Instruction Trace Block
1000: <i>divuw</i> 1004: <i>add</i> 1008: <i>or</i> 100C: <i>c.jalr</i>	iretire=7, iaddr=0x1000, itype=8

Retired	Instruction Trace Block
0940: <i>addi</i> 0944: <i>c.beq</i>	iretire=3, iaddr=0x0940, itype=4
0946: <i>c.bnez</i>	iretire=1, iaddr=0x0946, itype=5
0988: <i>lbu</i> 098C: <i>csrrw</i>	iretire=4, iaddr=0x0988, itype=0

2.3. Data Trace Interface requirements

This section describes in general terms the information which must be passed from the RISC-V hart to the trace encoder for the purposes of Data Trace, and distinguishes between what is mandatory, and what is optional.

If Data Trace is not needed in a system then there is no requirement for the RISC-V hart to supply any of the signals in [Section 2.4](#).

Data trace supports up to four data access types: load, store, atomic and CSR. Support for both atomic and CSR accesses are independently optional.

The signalling protocol can take one of two forms, depending on the needs of the RISC-V hart: *unified* or *split*. A hart is expected to provide one of these forms, not both simultaneously.

Unified is the simplest form, suitable for simpler, in-order harts. In this form, all information about a data access is signalled by the RISC-V hart in the same cycle. This is also the cycle that the associated data access instruction is reported as retired on the instruction trace interface, except for multi-memory access instructions - see [Section 2.4.1](#).

For harts with out of order or speculative execution capabilities, many loads may be in progress simultaneously, and this approach is not practical as it would require the hart to maintain a large amount of state relating to all the in-progress loads. For this reason, the interface also supports splitting loads, atomic and CSR accesses into two parts:

- The *request* phase provides all the information about the access that originates from the hart (address, size, etc.), and is valid in the same cycle the instruction retires, except for multi-memory access instructions;
- The *response* phase provides the load data and response status when it has been returned to the hart from the memory system.

The two parts of a split load are associated by use of a transaction ID.

For CSR accesses, the *request* and *response* phases should occur simultaneously.

2.4. Data Trace Interface

This section describes the interface between a RISC-V hart and the trace encoder that conveys the information described in the [Section 2.3](#). Signals are assigned to one of the following groups:

- M: Mandatory. The interface must include an instance of this signal;
- U: Unified. Mandatory for unified signalling;
- S: Split. Mandatory for split load signalling;

- O: Optional. The interface may include an instance of this signal.

All signals in M, U and O groups are only valid when **dretire** is high. Signals in the S group are valid as indicated in table [Table 12](#).

For harts that can retire a maximum of M data accesses per cycle, the implemented signal groups must be replicated M times. If fewer than M groups are required in a cycle, then lower numbered groups must be used first. For example, if there is one data access, use only group 0.

Table 12. Data interface signals

Signal	Group	Function
dretire	M	Data access retired (when high)
dtype [<i>dtype_width_p-1:0</i>]	M	Data access type. Encoding given in Table 13
daddr [<i>daddress_width_p-1:0</i>]	M	The data access address
dsize [<i>dsize_width_p-1:0</i>]	M	The data access size is 2^{dsize} bytes
data [<i>data_width_p-1:0</i>]	U	The data
iaddr_lsbs [<i>iaddr_lsbs_width_p-1:0</i>]	O	LSBs of the data access instruction address. Required if <i>retires_p</i> > 1
dblock [<i>dblock_width_p-1:0</i>]	O	Instruction block in which the data access instruction is retired. Required if there are replicated instruction block signals
lrid [<i>lrid_width_p-1:0</i>]	S	Load request ID. Valid when dretire is high. Ignored if dtype indicates a store.
lresp [<i>lresp_width_p-1:0</i>]	S	Load response: 0: None 1: reserved 2: Okay. Load successful; ldata valid 3: Error. Load failed; ldata not valid
lid [<i>lrid_width_p-1:0</i>]	S	Split Load ID. Valid when lresp is non-zero
sdata [<i>sdata_width_p-1:0</i>]	S	Store data. Valid when dretire is high and access is a store (dtype is 1 or 3), CSR (dtype is 4 - 6) or atomic (dtype is 8 - 14).
ldata [<i>ldata_width_p-1:0</i>]	S	Load data. Valid when lresp is non-zero

Table 13. Data access type (**dtype**) encoding

Value	Description
0	Load
1	Store
2	Multi-memory-access Load
3	Multi-memory-access Store
4	CSR read-write
5	CSR read-set
6	CSR read-clear
7	Compare and swap failure
8	Atomic swap
9	Atomic add
10	Atomic AND
11	Atomic OR
12	Atomic XOR
13	Atomic max
14	Atomic min
15	Conditional store failure

The maximum value of *dtype_width_p* is 4. However, if only loads and stores are supported, *dtype_width_p* can be 1. If Multi-memory access instructions are also required, *dtype_width_p* can be 2. If CSRs are supported but atomics are not, *dtype_width_p* can be 3.

Atomic and CSR accesses have load data and an operand (the actual stored data is obtained by applying the operator indicated by **dtype** to these values). When using unified signalling, both values are reported via **data**, with the operand data in the LSBs (occupying bits 0 through *data_width_p/2* - 1) and the load data in the MSBs (occupying bits *data_width_p/2* through *data_width_p* - 1). When using split signalling, the operand is reported via **sdata** and the load data via **ldata**.

Store conditional (SC) and Compare-and-swap (CAS) instructions both perform stores only if some condition is met. When a store is not performed, the trace decoder needs to be made aware in order to be able to correctly associate store packets in the data trace stream with the correct instruction.

- SC and CAS instructions where the condition is met and the store occurs are reported using **dtype** codes 1 and 8 respectively;
- SC and CAS instructions where the condition is not met and no store occurs are reported using **dtype** codes 15 and 7 respectively.

lrid_width_p is determined by the maximum number of loads that can be in progress simultaneously, such that at any one time there can be no more than one load in progress with a given ID.

iaddr_lsbs and **dblock** are provided to support filtering of which data accesses to trace based on their instruction address. This is best illustrated by considering the following instruction sequence:

1. load
2. <some non data access instruction>
3. load

4. <some non data access instruction>
5. <some non data access instruction>

Suppose the hart is capable of retiring up to 4 instructions in a cycle, via a single block. Instruction trace is enabled throughout, but the requirement is to collect data trace for the 1st load (instruction 1), and filtering is configured to match the address of this instruction only. However, information about instruction addresses is passed to the encoder at the block level, and the block boundaries are invisible to the trace decoder. For instruction trace, all instructions in a block are traced if any of the instructions in that block match the filtering criteria. That is fine for instruction trace - the address of the 1st and final traced instruction are output explicitly. There will be some fuzziness about precisely what those addresses will be depending on where the block boundaries fall, but this is not a concern as everything is always self-consistent.

However, that is not the case for data trace. Consider two scenarios:

- Case 1: 1st block contains instructions 1, 2, 3; second block contains 4, 5
- Case 2: 1st block contains instructions 1, 2; second block contains 3, 4, 5

The signals from the [Section 2.2](#) that provide details about the instruction block containing a data access instruction must be valid when `dretire` is active. Given this, the encoder knows the address of the 1st and last instructions in a block, but does not know precisely where in the block the data access is. In both cases, the first block matches the filtering criteria (it contains the address of instruction 1), and the second block does not. But if the encoder traced all the data accesses in the matching block, then in case 1 it would trace both instructions 1 and 3, whereas in the second case it would trace only instruction 1. The trace decoder has no visibility of the block boundaries so cannot account for this. It is expecting only instruction 1 to be traced, and so may misinterpret instruction 3. If this code is in a loop for example, it will assume that the 2nd traced load is in fact instruction 1 from the next loop iteration, rather than instruction 3 from this iteration.

Providing the LSBs of the data access instruction address allows the trace decoder to determine precisely whether the data access should be traced or not, and removes the dependency on the block sizes and boundaries. The number of bits required is one more bit than the number required to index within the block because blocks can start on any half-word boundary.

For harts that replicate the block signals to allow multiple blocks to retire per cycle it is also necessary to indicate which block each data access is associated with, so the encoder knows which block address to combine with the LSBs in order to construct the actual data access instruction address. 1 bit for 2 blocks per cycle, 2 bits for 4, and so on.

2.4.1. Multi-memory-access Instructions



This section introduces a normative change from the behavior described in the E-trace spec. This is necessary because the original behavior would result in incorrect N-Trace instruction trace. It could also result in misleading E-Trace instruction trace when one of these instructions traps part way through.

The previous requirement to report multi-access instructions as retired multiple times (once for each load or store) is withdrawn and deprecated for new designs. They should be reported as retired only once in the same way as any other instruction.

The Zcmp and Vector extensions include instructions that result in multiple loads or stores, and other future extensions may add other instructions with similar characteristics. It is not practical for the hart to provide information about all the loads or stores initiated by such an instruction simultaneously when the

instruction retires; it must be provided for each load or store in turn as it completes. This means that load/store information for these instructions is provided to the trace decoder before the instruction itself retires. This in turn means that **dretire** will be active when **iretire** is not, and this requires that the signals from the [Section 2.2](#) that provide details about the instruction that are normally defined to be valid when **iretire** is non-zero must also be valid when **dretire** is 1. The final load/store initiated by the instruction must be reported no later than when the instruction is reported as retired.

These multi-memory-access instructions may be interrupted part-way through their access sequence, or a trap may occur on one of the accesses. This can lead to difficulties correlating the data and instruction trace streams. Consider the following situation:

- Instruction "X" retires;
- A vector store that performs 4 stores starts executing;
- 2 stores are completed and then a trap occurs on the 3rd store;
- The trap handler includes a least one store;
- On returning from the trap handler, the vector store instruction resumes, performs the remaining stores and then retires.

The program flow decoded from the instruction trace stream will show that a trap occurred following instruction "X", with the vector store following the trap handler return. On the other hand, the data trace stream will have the 2 vector stores that occurred before the trap, then the stores and loads from the trap handler, then the remaining vector stores. It can be seen here that the order of the stores in the data trace stream does not match the order of store retirements in the instruction trace stream.

In order to allow the trace decoder to resolve ordering issues such as this, and correlate the loads and stores with the correct instruction, loads or stores from multi-memory-access instructions are identified using dedicated **dtype** encodings. On encountering these loads or stores in the data trace stream, the trace decoder must set these aside in a stack-like structure until a multi-access load or store instruction is encountered in the instruction trace stream. It can then correlate the appropriate number of the most recent multi-memory-access loads or stores with the instruction.

Some multi-access instructions restart from the beginning following a trap, and this complicates the correlation process, but such repeat loads or stores can be identified from their addresses. For example, a *cm.push* instruction that performs 4 stores that faults after the 2nd store will result in a sequence of 6 stores at addresses A, A+1, A, A+1, A+2, A+3. This will only happen when the multi-access load/store retires immediately after a **ret*, and the number of repeated accesses can be determined from the addressing pattern.

Special consideration is also needed if data trace is enabled and tracing starts on the cycle that an MMA instruction retires. The start could be initiated by any cause (software, a *trace-on* trigger, filter qualification, etc.) When tracing starts, the instructions retired in that cycle along with any loads or stores reported in that cycle are included in the trace stream. The final load or store in an MMA instruction may occur at the same time that the instruction retires, or may be earlier. Enabling trace on the cycle that the MMA retires means there may be one or none of the loads or stores of that instruction included in the data trace stream, and this indeterminism presents difficulties for correlating the instruction and data trace streams during decoding. To avoid this, if an MMA instruction is in progress (MMA loads or stores observed via **dretire** and **dtype** but instruction not retired), tracing must not be enabled until after the MMA retires. The effect of this will be to delay the start of trace by one cycle. But it ensures all retired MMA instructions in the instruction trace stream will be accompanied by a full set of loads/stores in the data trace stream.

Chapter 3. Hart to Trace Encoder interface extensions

This chapter specifies the following standard extensions to the base Hart to Trace Interface.

Specification	Version	Status
Trace State	0.9	Fast-track

3.1. Trace State

The **halted** signal defined in [Optional sideband encoder input signals](#) provides a mechanism to inhibit tracing whilst the hart is in Debug mode. However, since this was defined, other conditions that require similar behaviour have been identified:

- Inhibiting trace for security reasons;
- Pausing trace for an unbounded amount of time.

To accomplish this, the **halted** signal is replaced with the multi-bit **trcstate** signal defined below:

Table 14. Optional sideband encoder input signal for trace state

Signal	Group	Function
trcstate[1:0]	O	00: Tracing uninhibited by the hart 01: Inhibit trace as the hart is in debug mode. 10: Inhibit trace for security reasons. 11: Unbounded wait.

When set to any non-zero value, the encoder will output a packet containing the information the decoder requires in order to determine the final instruction retired up to this point (e.g. the address of that instruction, or the instruction count since the last reported instruction). If trace is inhibited (but not paused), the encoder will then also indicate that tracing has stopped.

Upon return to zero, the encoder may start tracing again (subject to any other filtering), commencing with a synchronization packet.

If there are simultaneously multiple different reasons for the hart to drive a non-zero value, it should prioritize the lowest numerical value.

Note: If neither **halted** nor **trcstate** are provided, it is strongly recommended that Debug mode can be signalled via the **priv** signal. This will allow tracing in Debug mode to be controlled via the optional filtering capabilities.

3.1.1. External Debug Security (sdsec)

The External Debug Security specification introduces the concept of the logical 'sec_inhibit' signal, which when active requires trace to be inhibited. This can be achieved by setting **trcstate** to 10 when 'sec_inhibit' is true.

3.1.2. Unbounded Wait

Several instructions can cause the hart to wait for some condition before retiring. These can be further classified as bounded or unbounded:

- *bounded wait* instructions such as PAUSE or Wait on Reservation Set with short time-out (WRS.STO), where there is a maximum wait time;
- *unbounded wait* instructions such as Wait for Interrupt (WFI) or Wait on Reservation Set with no time-out (WRS.NTO), where there is no upper limit on the duration of the wait time.

An unbounded wait can result in a long pause in activity, and in some cases (possibly as the result of a software bug), the pause may be indefinite.

Providing a mechanism for the trace encoder to indicate that an unbounded wait has been entered allows the user to be informed of this, and if timestamps are enabled gives an indication of the time of entry. Combined with a timestamped sync packet following exit from the wait, this allows the user to determine how long the hart spent in the wait state.

Without this mechanism, unbounded wait instructions have no direct impact on the trace stream (they are treated as ordinary instructions with **itype** = 0 when they retire). If an indefinite wait occurs, the trace message stream simply stops without any indication as to why.

Unbounded wait instructions may also disable the hart clock, and by extension the clock used by the trace encoder. For encoders that utilize partial timestamps, any accrued branch history must be reported within a maximum time from the last packet in order to maintain timestamp integrity, and this cannot be done if the clock is stopped. Outputting any such history when **trcstate** becomes 11 prior to disabling the clock avoids this potential issue.

3.1.3. Backwards compatibility considerations

- A hart which supports **halted** can be connected to a Trace Encoder that supports **trcstate** by wiring **halted** to **trcstate[0]**, and tying **trcstate[1]** to 0.
- A hart which supports **trcstate** can be connected to a Trace Encoder that supports **halted** by connecting the OR of both bits of the hart's **trcstate** output to the encoder's **halted** input.